

KI-Systeme und Entwicklungslinie

Master-Dossier

Remote, Miguel, Routing, Memory, Voice, ANIMA und Genesis

Remote WhatsApp-to-Agent Command Loop

Ich wollte von unterwegs mit meinem Handy Arbeit an meinen Laptop übergeben koennen, ohne daraus eine gefaehrliche direkte Shell zu machen. Die WhatsApp-Nachricht sollte erst verstanden, in einen Arbeitsauftrag übersetzt, ausgeführt, bewertet und dann sauber zurückgemeldet werden.

- Das stärkste Signal ist nicht der Chatkanal, sondern die sichere Übersetzung von Alltagssprache in einen geprüften Arbeitsauftrag: Eingang, Interpretation, Ausführung, Bewertung und erst danach Rückmeldung.
- Persönlicher Prototyp bzw. entwickeltes System, keine Behauptung eines produktionsgehärteten Sicherheitsprodukts.

Was ich mir dabei gedacht habe

Was ich mir dabei gedacht habe, war einfach, aber technisch anspruchsvoll: Ich wollte nicht am Laptop sitzen müssen und trotzdem sinnvoll Arbeit an ihn übergeben können, ohne mein Handy zu einer blinden Remote-Shell zu machen. Der interessante Teil war die Vertrauensgrenze. Eine Nachricht musste erst zum Arbeitsauftrag werden, ausgeführt und danach bewertet werden, bevor sie zu mir zurückkommt.

Was ich gebaut habe

- Ich habe Fernzugriff als Autoritäts- und Sicherheitsproblem behandelt, nicht nur als Komfortfunktion.
- Ich habe die Front-Agenten-Schicht von der tool-reichen Ausführungsschicht getrennt.
- Ich habe mobile Sprache/Chat in strukturierte Arbeit mit Erfolgskriterien übersetzt.

Mechanik im Detail

- Der wertvolle Teil ist nicht WhatsApp selbst. WhatsApp ist nur der reibungsarme Eingang. Der harte Teil ist die Übersetzung: eine lockere Nachricht vom Handy wird zu einem strukturierten Arbeitsauftrag mit Ziel, Komplexität, Modellhinweis und Erfolgskriterien.
- Das Dual-Brain-Muster trennt die Rolle, die versteht und bewertet, von der Rolle, die mit Tools ausführt. Dadurch wird eine Handy-Nachricht nicht zu einem ungeprüften Direktbefehl.
- Die Antwort kommt bewusst erst nach der Bewertung zurück. Das ist näher an Supervisor/Executor als an einem normalen Chatbot.

Architektur

- WhatsApp oder Audio kommt in der OpenClaw-Schicht an.
- Ein Watcher normalisiert Ereignisse, dedupliziert Nachrichten und behandelt Medien.
- MiguelAgent macht daraus Ziel, Komplexität, Arbeitsauftrag, Erfolgskriterien und Modell-Hinweis.
- Die Ausführungsseite nutzt Miguel/EXOVIUM-Tools und Modellrouting.
- Eine Qualitätsschleife prüft, ob das Ergebnis wirklich fertig ist.
- Erst danach wird die Antwort wieder an WhatsApp geschickt.

Evidenzcluster

- Workflow-Notizen zu WhatsApp-Event-Erkennung, Audio-Transkription, strukturierten Arbeitsaufträgen, Modellhinweisen und Quality-Loop-Schwellen.
- Pipeline-Notizen zur Kette WhatsApp -> Gateway -> Watcher -> MiguelAgent -> Ausführung/Bewertung/Formatierung -> Antwort.
- Python-Watcher- und MiguelAgent-Module mit Event-Parsing, Medienhandling, Deduplication, Arbeitsauftragsausführung, Evaluator-Loop und Antwortformatierung.
- Entscheidungsnotizen zu schneller Bestätigung, Smart Routing und erfolgreichen End-to-End-Tests.

Claim-zu-Evidenz-Karte

Die öffentliche Version soll stark wirken, weil sie begrenzt ist, nicht weil sie übertreibt.

Claim	Evidenz	Grenze
Das stärkste Signal ist nicht der Chatkanal, sondern die sichere Übersetzung von Alltagssprache in einen geprüften Arbeitsauftrag: Eingang, Interpretation, Ausführung, Bewertung und erst danach Rückmeldung.	Workflow-Notizen zu WhatsApp-Event-Erkennung, Audio-Transkription, strukturierten Arbeitsaufträgen, Modellhinweisen und Quality-Loop-Schwellen.; Pipeline-Notizen zur Kette WhatsApp -> Gateway -> Watcher -> MiguelAgent -> Ausführung/Bewertung/Formatierung -> Antwort.	Persönlicher Prototyp bzw. entwickeltes System, keine Behauptung eines produktionsgehärteten Sicherheitsprodukts.
Die Arbeit ist wertvoll, weil sie Mechanik zeigt, nicht nur Oberfläche.	WhatsApp oder Audio kommt in der OpenClaw-Schicht an.; Ein Watcher normalisiert Ereignisse, dedupliziert Nachrichten und behandelt Medien.	Frische öffentliche Demos oder Benchmarks wären die nächste Beweisschicht.

Senior-Level-Signal

Das stärkste Signal ist nicht der Chatkanal, sondern die sichere Übersetzung von Alltagssprache in einen geprüften Arbeitsauftrag: Eingang, Interpretation, Ausführung, Bewertung und erst danach Rückmeldung.

- Autoritätsgrenzen: Der Remote-Kanal ist bequem, aber das System behandelt ihn als riskante Kontrollfläche.
- Operatives Denken: schnelle Bestätigung, Deduplication, Medienhandling und finale Rückmeldung sind getrennte Fehlerpunkte.
- Evaluator-Loop-Intuition: Das System führt nicht nur aus, sondern prüft, ob das Ergebnis den ursprünglichen Auftrag erfüllt.

Skeptische Reviewer-Fragen

- Was ist hier wirklich gebaut und was ist noch Design? Antwort: Status und Grenze stehen im Dossier direkt beim Fall Remote WhatsApp-to-Agent Command Loop.
- Welche private Evidenz wurde bewusst nicht veröffentlicht? Antwort: lokale Pfade, Credentials, private Kanaldetails und vertrauliche Rohtexte bleiben draußen.
- Warum ist das relevant? Antwort: Der Fall zeigt einen wiederverwendbaren Baustein für Agenten, Tool-Nutzung, Memory, Routing, Validierung oder High-Stakes-Governance.
- Was wäre der nächste belastbare Beweis? Antwort: synthetische Demo, Audit Trace, Benchmark oder externe Review, je nach Fall.

Lernpunkte und Grenzen

Persönlicher Prototyp bzw. entwickeltes System, keine Behauptung eines produktionsgehärteten Sicherheitsprodukts.

- Schnell bestaetigen, langsam und kontrolliert arbeiten, erst nach Bewertung antworten.
- Remote-Steuerung braucht Deduplikation, Unterbrechbarkeit, Fortschritt und Audit-Spuren.
- Private Kanal-IDs und Zugangsdaten gehoeren nie in ein öffentliches Portfolio.

Nächste Härtung / nächster Projektschritt

- Öffentlichen Demo-Modus mit synthetischen Daten bauen.
- Befehle nach Risiko und Bestaetigungsbedarf klassifizieren.
- Tool-Aufrufe, Modellwahl und Bewertung als sichere Audit-Events speichern.
- Einen öffentlichen synthetischen Demo-Flow bauen, in dem ein Mock-WhatsApp-Event zu einem Arbeitsauftrag wird, ein Fake-Executor läuft und ein Evaluator freigibt oder Nacharbeit verlangt.
- Jeden Übergang loggen: Eingang, Normalisierung, Work Order, Tool Plan, Executor Result, Evaluator Result und Antwort.
- Permission-Klassen für Read-only, Write, Shell, Netzwerk und irreversible Aktionen hinzufügen.

Miguel Core Unified Daemon

Ich wollte meine vielen einzelnen Agenten-, Voice-, Memory- und Tool-Skripte nicht als lose Sammlung behalten. Miguel Core war mein Versuch, daraus eine beobachtbare lokale Kontrollschicht zu machen.

- Zeigt, dass ich viele einzelne KI-Experimente nicht nur gesammelt, sondern zu einer lokalen Kontrollschicht mit Routen, State, Health, Worker-Delegation und Grenzen verdichtet habe.
- Lokale experimentelle Kontrollschicht, nicht als fertiges kommerzielles Produkt formuliert.

Was ich mir dabei gedacht habe

Miguel Core entstand aus einer sehr praktischen Frustration: Ich hatte zu viele starke Teile als einzelne Skripte und Services. Ich wollte daraus eine lokale Kontrollschicht machen, die beobachtbar, routbar und verständlicher ist.

Was ich gebaut habe

- Ich denke Agenten nicht als einzelne Prompts, sondern als Betriebssystem-nahe Services.
- Ich kann verstreute Prototypen in eine zentrale Kontrollflaeche zusammenfuehren.
- Ich habe frueh ueber Health, State, Tool-Oberflaechen und Autonomiegrenzen nachgedacht.

Mechanik im Detail

- Miguel Core lässt sich am stärksten als lokale Agenten-Betriebsschicht erklären. Chat, Streaming, Voice, State, Memory, Prediction, Modellrouting, Orchestrierung und Tool-Endpunkte werden in eine beobachtbare Kontrollschicht gezogen.
- Die Route Surface ist wichtig, weil sie zeigt, dass die Arbeit von Einzelskripten in Richtung System mit Health, State, Conversation Persistence und Worker Delegation gegangen ist.
- Startup- und Handover-Dokumente gehören zur Evidenz: Sie definieren Boot Checks, Betriebsmodi, Quality Anchors, Autonomy Rings und State Probes.

Architektur

- Ein Python-Daemon bundelt Chat, Streaming, Voice, State, Vault, Knowledge und Tools.
- Routen verbinden Memory Broker, CASS, Vault Watcher, Prediction, Metacognition und macOS Bridge.
- Orchestra delegiert Aufgaben an Worker und verfolgt Status, Timeout, Ergebnis und Abbruch.
- Start- und Handover-Dateien dokumentieren Modellrouting, Boot Checks und Autonomie-Ringe.

Evidenzcluster

- Miguel-Core-Daemon als Konsolidierung früherer Daemon- und Command-Flächen in eine lokale Kontrollschicht.
- Route Map für Chat, Streaming, Voice, State, Vault, Knowledge, Prediction, Cascade, Agents, Orchestra, macOS/Tool-Control, Conversations und Health.
- Orchestra-Worker-Modul mit asynchroner Delegation, Timeouts, Cancellation, Status und Result Capture.
- Startup- und Handover-Dokumente mit Boot-Probes, Routing Policies, Quality Gates, Betriebsmodi und Autonomy Rings.

Claim-zu-Evidenz-Karte

Die öffentliche Version soll stark wirken, weil sie begrenzt ist, nicht weil sie übertreibt.

Claim	Evidenz	Grenze
Zeigt, dass ich viele einzelne KI-Experimente nicht nur gesammelt, sondern zu einer lokalen Kontrollschicht mit Routen, State, Health, Worker-Delegation und Grenzen verdichtet habe.	Miguel-Core-Daemon als Konsolidierung früherer Daemon- und Command-Flächen in eine lokale Kontrollschicht.; Route Map für Chat, Streaming, Voice, State, Vault, Knowledge, Prediction, Cascade, Agents, Orchestra, macOS/Tool-Control, Conversations und Health.	Lokale experimentelle Kontrollschicht, nicht als fertiges kommerzielles Produkt formuliert.
Die Arbeit ist wertvoll, weil sie Mechanik zeigt, nicht nur Oberfläche.	Ein Python-Daemon bundelt Chat, Streaming, Voice, State, Vault, Knowledge und Tools.; Routen verbinden Memory Broker, CASS, Vault Watcher, Prediction, Metacognition und macOS Bridge.	Frische öffentliche Demos oder Benchmarks wären die nächste Beweisschicht.

Senior-Level-Signal

Zeigt, dass ich viele einzelne KI-Experimente nicht nur gesammelt, sondern zu einer lokalen Kontrollschicht mit Routen, State, Health, Worker-Delegation und Grenzen verdichtet habe.

- Architekturkonsolidierung: Fragmentierung erkennen und viele Flächen in eine kontrollierbare Schicht bringen.
- Operative Reife: Boot Probes und Handover-Dokumente zeigen, dass das System betrieben werden sollte, nicht nur als Idee existiert.
- Grenzreife: Autonomy Rings trennen, was das System darf, von dem, was Nutzerfreigabe braucht.

Skeptische Reviewer-Fragen

- Was ist hier wirklich gebaut und was ist noch Design? Antwort: Status und Grenze stehen im Dossier direkt beim Fall Miguel Core Unified Daemon.
- Welche private Evidenz wurde bewusst nicht veröffentlicht? Antwort: lokale Pfade, Credentials, private Kanaldetails und vertrauliche Rohtexte bleiben draussen.
- Warum ist das relevant? Antwort: Der Fall zeigt einen wiederverwendbaren Baustein für Agenten, Tool-Nutzung, Memory, Routing, Validierung oder High-Stakes-Governance.
- Was wäre der nächste belastbare Beweis? Antwort: synthetische Demo, Audit Trace, Benchmark oder externe Review, je nach Fall.

Lernpunkte und Grenzen

Lokale experimentelle Kontrollschicht, nicht als fertiges kommerzielles Produkt formuliert.

- Eine Route zu haben ist nicht dasselbe wie eine Route produktionsreif zu haerten.
- Lokale Systeme brauchen trotzdem Beobachtbarkeit, klare Rechte und Fehlergrenzen.
- Autonomie-Ringe sind eine hilfreiche Sprache für sichere und unsichere Aktionen.

Nächste Härtung / nächster Projektschritt

- Stabile Public-API aus dem breiten internen Daemon herausloesen.
- Endpoint-Rechte und Risiko-Labels maschinenlesbar dokumentieren.
- Health Reports für alle Hauptmodule erzeugen.
- Eine öffentliche Demo-Route-Map mit privaten Aktionen als sichere Stubs zeigen.
- Endpoint-Metadaten ergänzen: Permission, betroffene Daten, Modell, Timeout und Audit Trace.
- Einen synthetischen Health Report veröffentlichen, der pro Route den Bereitschaftszustand ausweist.

Multi-Model Cascade Router

Ich wollte nicht alles blind durch ein Modell schicken. Der Router behandelt Modelle als Ressourcen mit Staerken, Kosten, Quoten, Ausfaellen, Qualität und Datenschutzkontext.

- Zeigt Modell-Governance in der Praxis: Modelle werden nach Kosten, Qualität, Privacy, Quoten, Ausfallrisiko und Eskalationslogik gewählt, nicht nach Bauchgefühl.
- Exakte Kostenersparnis nur dann behaupten, wenn neue Messlogs sie belegen.

Was ich mir dabei gedacht habe

Ich wollte Modellwahl nicht als Geschmack behandeln. Routing sollte Teil der Architektur sein: günstig, wenn möglich; stark, wenn nötig; lokal, wenn Privacy zählt; fallback-fähig, wenn ein Provider ausfällt.

Was ich gebaut habe

- Ich habe Modellwahl als Governance-System behandelt, nicht als Geschmacksfrage.
- Ich habe Kosten, Latenz, Quoten, Qualität und Fallbacks im selben System gedacht.
- Ich habe lokale und API-basierte Modelle in einer Routing-Logik verbunden.

Mechanik im Detail

- Der Router behandelt Modelle als Infrastrukturressourcen mit Stärken, Kosten, Quoten, Cooldowns, Privacy-Profilen und Ausfallmodi.
- Die Task-Strategie entscheidet, ob Geschwindigkeit, Qualität, kostenlose Modelle, lokale Ausführung, Code-Fähigkeit oder bezahlte Eskalation wichtiger ist.
- Der Sicherheitswert liegt in graceful degradation: Wenn ein Modell oder eine Ebene ausfällt, existiert eine Policy für den nächsten Weg.

Architektur

- Task-Strategien wählen schnell, hochwertig, gratis, lokal, Code, kreativ oder bulk.
- Lokale Modelle und kostenlose APIs werden dort bevorzugt, wo es sinnvoll ist.
- Quoten, Cooldowns und Ausfälle blockieren schlechte Routen.
- Qualitätsgates entscheiden, ob eskaliert wird.
- Mehrere freie Modelle können gegeneinander laufen, langsamere Calls werden abgebrochen.

Evidenzcluster

- AI-Router mit Provider-Registry für lokale Modelle, freie APIs, bezahlte/CLI-Modelle und LiteLLM-gestützte Calls.
- Cascade-Logik mit lokalen/freien/bezahlten Ebenen, Quoten, Cooldowns, Quality Gates, Free-Model-Racing, Cancellation und Budgetgrenzen.
- LiteLLM-Konfiguration mit getrennten lokalen, freien API- und bezahlten Ebenen.
- Architekturnotizen zu Routing-Prinzipien, Qualitätsbewertung, Budgethandling und fallbacks bei Quoten/Ausfällen.

Claim-zu-Evidenz-Karte

Die öffentliche Version soll stark wirken, weil sie begrenzt ist, nicht weil sie übertreibt.

Claim	Evidenz	Grenze
Zeigt Modell-Governance in der Praxis: Modelle werden nach Kosten, Qualität, Privacy, Quoten, Ausfallrisiko und Eskalationslogik gewählt, nicht nach Bauchgefühl.	AI-Router mit Provider-Registry für lokale Modelle, freie APIs, bezahlte/CLI-Modelle und LiteLLM-gestützte Calls.; Cascade-Logik mit lokalen/freien/bezahlten Ebenen, Quoten, Cooldowns, Quality Gates, Free-Model-Racing, Cancellation und Budgetgrenzen.	Exakte Kostenersparnis nur dann behaupten, wenn neue Messlogs sie belegen.
Die Arbeit ist wertvoll, weil sie Mechanik zeigt, nicht nur Oberfläche.	Task-Strategien wählen schnell, hochwertig, gratis, lokal, Code, kreativ oder bulk.; Lokale Modelle und kostenlose APIs werden dort bevorzugt, wo es sinnvoll ist.	Frische öffentliche Demos oder Benchmarks wären die nächste Beweisschicht.

Senior-Level-Signal

Zeigt Modell-Governance in der Praxis: Modelle werden nach Kosten, Qualität, Privacy, Quoten, Ausfallrisiko und Eskalationslogik gewählt, nicht nach Bauchgefühl.

- Provider-neutrales Denken: Kein einzelnes Modell wird als magisch behandelt.
- Governance-Denken: Kosten, Qualität, Privacy und Quoten gehören zur Modellwahl.
- Messpfad: Outcome Logging schafft die Grundlage für spätere empirische Routing-Entscheidungen.

Skeptische Reviewer-Fragen

- Was ist hier wirklich gebaut und was ist noch Design? Antwort: Status und Grenze stehen im Dossier direkt beim Fall Multi-Model Cascade Router.
- Welche private Evidenz wurde bewusst nicht veröffentlicht? Antwort: lokale Pfade, Credentials, private Kanaldetails und vertrauliche Rohtexte bleiben draussen.
- Warum ist das relevant? Antwort: Der Fall zeigt einen wiederverwendbaren Baustein für Agenten, Tool-Nutzung, Memory, Routing, Validierung oder High-Stakes-Governance.
- Was wäre der nächste belastbare Beweis? Antwort: synthetische Demo, Audit Trace, Benchmark oder externe Review, je nach Fall.

Lernpunkte und Grenzen

Exakte Kostenersparnis nur dann behaupten, wenn neue Messlogs sie belegen.

- Ein guter Agent muss elegant schlechter werden koennen, wenn Modelle ausfallen.
- Qualitätsgates sollten Eskalation auslösen, nicht Modell-Prestige.
- Outcome-Logging ist der Anfang empirischer Modell-Governance.

Nächste Härtung / nächster Projektschritt

- Frische Benchmarks ueber repraesentative Aufgabenklassen laufen lassen.
- Aggregierte Kosten/Latenz/Qualität ohne Geheimnisse veröffentlichen.
- Policy-Labels für privat, öffentlich, billig, schnell und hochriskant einfuehren.
- Benchmark mit synthetischen Aufgaben: Code Repair, Research Summary, Planung, Long-Context Extraction und Low-Cost-Bulk-Routing.
- Aggregierte Latenz/Kosten/Qualität ohne private Provider-Details veröffentlichen.
- Policy-Labels ergänzen, damit hochriskante oder private Aufgaben auf sicherere Routen gezwungen werden können.

Memory, Vault, CASS und Smart Memory MCP

Ich wollte Agenten nicht nur groessere Kontextfenster geben, sondern echtes Arbeitsgedaechtnis: roh, zusammengefasst, prozedural, widerspruchsbewusst, veraltetheitsbewusst und wieder abrufbar.

- Zeigt Verständnis für Agenten-Zuverlässigkeit: Memory ist nicht nur Kontext, sondern Provenance, Widerspruchsbehandlung, Staleness, Zugriffshistorie und prozedurales Lernen.
- Praktische Memory-Infrastruktur, keine Behauptung menschenähnlichen Bewusstseins.

Was ich mir dabei gedacht habe

Die Memory-Arbeit entstand aus der Beobachtung, dass Agenten nicht nur mehr Kontext brauchen. Sie brauchen Memory-Hygiene: Rohfakten, Zusammenfassungen, prozedurales Lernen, Widersprüche, Veraltetheit und Provenance.

Was ich gebaut habe

- Ich habe Memory als Governance-Problem behandelt, nicht als Notizablage.
- Ich habe Rohdaten, Zusammenfassungen und Regeln getrennt.
- Ich habe Widerspruch, Veraltung und Herkunft als zentrale Risiken erkannt.

Mechanik im Detail

- Die Memory-Arbeit ist am stärksten, wenn sie als Reliability Infrastructure erklärt wird. Sie trennt rohe episodische Records von komprimiertem Working Memory und prozeduralen Regeln.
- Broker/Broadcast reagieren auf Modul-Isolation: wichtige Fakten, zentrale Notizen und Widersprüche müssen zwischen Subsystemen wandern können.
- Vault Watcher und Smart Memory MCP machen Memory zu mehr als einem Ordner: Indexierung, Recency, Usefulness, Access-Metadaten, Graph-Kontext und Stale-Information-Handling.

Architektur

- CASS trennt episodische Rohdaten, Arbeitsgedächtnis und prozedurale Regeln.
- Memory Broker verteilt wichtige Fakten und markiert leichte Widersprüche.
- Vault Watcher denkt Dateiaenderungen, inkrementelles Indexing, Graph Updates und Staleness.
- Smart Memory MCP macht Lernen, Recall, Stats, Muster und Vorschläge als lokale Tools verfügbar.

Evidenzcluster

- Memory Broker mit Broadcast-Schema, Write Logs, Weight/Centrality Scoring und leichter Widerspruchserkennung.
- CASS-Pipeline mit episodischem Rohgedächtnis, Working Summaries, prozeduralen Regeln, Confidence, Decay und SQLite-Persistenz.
- Vault-Watcher-Design mit File-Change-Detection, inkrementeller Indexierung, Graph Updates, Staleness Propagation und Forgetting Score.
- Smart Memory MCP mit Learn/Recall/Stats/Pattern/Suggestion-Tools und Retrieval-Metadaten.

Claim-zu-Evidenz-Karte

Die öffentliche Version soll stark wirken, weil sie begrenzt ist, nicht weil sie übertreibt.

Claim	Evidenz	Grenze
Zeigt Verständnis für Agenten-Zuverlässigkeit: Memory ist nicht nur Kontext, sondern Provenance, Widerspruchsbehandlung, Staleness, Zugriffshistorie und prozedurales Lernen.	Memory Broker mit Broadcast-Schema, Write Logs, Weight/Centrality Scoring und leichter Widerspruchserkennung.; CASS-Pipeline mit episodischem Rohgedächtnis, Working Summaries, prozeduralen Regeln, Confidence, Decay und SQLite-Persistenz.	Praktische Memory-Infrastruktur, keine Behauptung menschenähnlichen Bewusstseins.

Claim	Evidenz	Grenze
Die Arbeit ist wertvoll, weil sie Mechanik zeigt, nicht nur Oberfläche.	CASS trennt episodische Rohdaten, Arbeitsgedächtnis und prozedurale Regeln.; Memory Broker verteilt wichtige Fakten und markiert leichte Widersprüche.	Frische öffentliche Demos oder Benchmarks wären die nächste Beweisschicht.

Senior-Level-Signal

Zeigt Verständnis für Agenten-Zuverlässigkeit: Memory ist nicht nur Kontext, sondern Provenance, Widerspruchsbehandlung, Staleness, Zugriffshistorie und prozedurales Lernen.

- Kontextintegrität: Zusammenfassungen sind nützlich, aber gefährlich, wenn sie Rohbelege ersetzen.
- Widerspruchsbehandlung: Konflikte werden nicht einfach überschrieben.
- Agent-Safety-Relevanz: Stale Memory kann zu falschen Aktionen, falscher Sicherheit und Zielabweichung führen.

Skeptische Reviewer-Fragen

- Was ist hier wirklich gebaut und was ist noch Design? Antwort: Status und Grenze stehen im Dossier direkt beim Fall Memory, Vault, CASS und Smart Memory MCP.
- Welche private Evidenz wurde bewusst nicht veröffentlicht? Antwort: lokale Pfade, Credentials, private Kanaldetails und vertrauliche Rohtexte bleiben draußen.
- Warum ist das relevant? Antwort: Der Fall zeigt einen wiederverwendbaren Baustein für Agenten, Tool-Nutzung, Memory, Routing, Validierung oder High-Stakes-Governance.
- Was wäre der nächste belastbare Beweis? Antwort: synthetische Demo, Audit Trace, Benchmark oder externe Review, je nach Fall.

Lernpunkte und Grenzen

Praktische Memory-Infrastruktur, keine Behauptung menschenähnlichen Bewusstseins.

- Agenten werden gefährlich oder nutzlos, wenn sie veraltete Zusammenfassungen für Wahrheit halten.
- Widersprüche sollen erhalten und sichtbar gemacht werden, nicht still überschrieben.
- Retrieval braucht mehr als Textähnlichkeit: Nützlichkeit, Zeit und Vertrauen zählen.

Nächste Härtung / nächster Projektschritt

- Synthetische Public-Memory-Demo bauen.
- Retrieval und Widerspruchserkennung auf nicht-privaten Daten benchmarken.
- Memory-Provenance in Agentenantworten anzeigen.
- Einen synthetischen Vault mit absichtlichen Widersprüchen und veralteten Fakten bauen.
- Zeigen, wie der Agent Episoden, Working Summaries und prozedurale Regeln mit Provenance abruft.
- Retrieval-Qualität und Widerspruchserkennung auf öffentlichen Daten benchmarken.

Voice Interface and Spoken Tool Use

Ich wollte mit meinem System sprechen koennen, aber Sprache ist für Tools riskant: sie ist schnell, ungenau und kann trotzdem Aktionen auslösen. Deshalb brauchte ich Muting, Tool-Grenzen, Fallbacks und Schutz gegen gefährliche Befehle.

- Zeigt eine sicherheitssensible Mensch-Maschine-Fläche: Voice macht Tool-Nutzung extrem direkt, deshalb müssen Mute, Bestätigung, Shell-Grenzen, Timeouts und Privacy mitgebaut werden.
- Terminal/Prototyp und Planungsstand sauber trennen; Wake-Word/macOS-Teile nur behaupten, wenn frisch getestet.

Was ich mir dabei gedacht habe

Voice war spannend, weil sich das System dadurch direkt anfühlte. Genau deshalb ist es aber riskant. Wenn ich spreche und der Computer handelt, dann sind Mute, Bestätigung, Shell-Grenzen und Privacy kein Zubehör, sondern Teil des Produkts.

Was ich gebaut habe

- Ich habe Voice als tool-fähige Kontrolloberfläche gebaut, nicht nur als Chat-Spielerei.
- Ich habe Datenschutz, Bestätigung und Unterbrechbarkeit von Anfang an mitgedacht.
- Ich habe hochwertige Stimmen mit praktischer Tool-Nutzung verbunden.

Mechanik im Detail

- Das Voice-System ist nicht nur Speech-to-Text. Gesprochene Befehle können Tool Calls, Vault-Operationen, Daemon Calls und begrenzte Shell-Ausführung auslösen.
- Damit ist Voice eine sicherheitssensible Fläche. Mute, Fallbacks, Filter für gefährliche Befehle, Timeouts und Privacy-Grenzen sind Kernarchitektur.
- Der spätere Plan zu Wake Word, lokaler STT, hochwertiger TTS und Barge-in zeigt Bewusstsein für Latenz, Privacy und Unterbrechbarkeit.

Architektur

- Python streamt Mikrofon-Audio in Gemini Live.
- Tools erlauben Vault Search/Read/Write, Deep Thinking, Brain Status und begrenzte Shell.
- Gefährliche Shell-Muster werden blockiert, Ausführung bekommt Timeouts.
- Spätere Planung umfasst Wake Word, lokale STT, Hybrid-TTS, Barge-in und Privacy-Grenzen.

Evidenzcluster

- Voice-Source mit Mikrofonstreaming, Gemini-Live-Konfiguration, Tool Declarations, Dispatcher, Vault Tools, Deep-Think-Route, begrenzter Shell und Fallback.
- Voice-Pipeline-Notizen zu Wake Word, lokaler STT, hybrider TTS, Barge-in, Latenzbudget und Privacy-Grenzen.
- VoiceBridge-Notizen zur Trennung von Browser/Orb-STT und lokaler Daemon-Antwortlogik.
- Sicherheitsrelevante Quellabschnitte zu Mute-Verhalten, Filterung gefährlicher Befehle, Timeouts und Tool-Dispatch.

Claim-zu-Evidenz-Karte

Die öffentliche Version soll stark wirken, weil sie begrenzt ist, nicht weil sie übertreibt.

Claim	Evidenz	Grenze
Zeigt eine sicherheitssensible Mensch-Maschine-Fläche: Voice macht Tool-Nutzung extrem direkt, deshalb müssen Mute, Bestätigung, Shell-Grenzen, Timeouts und Privacy mitgebaut werden.	Voice-Source mit Mikrofonstreaming, Gemini-Live-Konfiguration, Tool Declarations, Dispatcher, Vault Tools, Deep-Think-Route, begrenzter Shell und Fallback.; Voice-Pipeline-Notizen zu Wake Word, lokaler STT, hybrider TTS, Barge-in, Latenzbudget und Privacy-Grenzen.	Terminal/Prototyp und Planungsstand sauber trennen; Wake-Word/macOS-Teile nur behaupten, wenn frisch getestet.
Die Arbeit ist wertvoll, weil sie Mechanik zeigt, nicht nur Oberfläche.	Python streamt Mikrofon-Audio in Gemini Live.; Tools erlauben Vault Search/Read/Write, Deep Thinking, Brain Status und begrenzte Shell.	Frische öffentliche Demos oder Benchmarks wären die nächste Beweisschicht.

Senior-Level-Signal

Zeigt eine sicherheitssensible Mensch-Maschine-Fläche: Voice macht Tool-Nutzung extrem direkt, deshalb müssen Mute, Bestätigung, Shell-Grenzen, Timeouts und Privacy mitgebaut werden.

- Realismus im Human Interface: Voice senkt Reibung so stark, dass Authority Checks stärker werden müssen.
- Privacy-Realismus: Always-listening-Systeme sollten lokal und mit klaren Grenzen arbeiten.
- Safety-Realismus: Barge-in ist Kontrollfunktion, kein Luxus.

Skeptische Reviewer-Fragen

- Was ist hier wirklich gebaut und was ist noch Design? Antwort: Status und Grenze stehen im Dossier direkt beim Fall Voice Interface and Spoken Tool Use.
- Welche private Evidenz wurde bewusst nicht veröffentlicht? Antwort: lokale Pfade, Credentials, private Kanaldetails und vertrauliche Rohtexte bleiben draußen.
- Warum ist das relevant? Antwort: Der Fall zeigt einen wiederverwendbaren Baustein für Agenten, Tool-Nutzung, Memory, Routing, Validierung oder High-Stakes-Governance.
- Was wäre der nächste belastbare Beweis? Antwort: synthetische Demo, Audit Trace, Benchmark oder externe Review, je nach Fall.

Lernpunkte und Grenzen

Terminal/Prototyp und Planungsstand sauber trennen; Wake-Word/macOS-Teile nur behaupten, wenn frisch getestet.

- Voice senkt Reibung beim Auslösen und muss deshalb Reibung beim Risiko erhöhen.
- Barge-in ist ein Sicherheitsfeature, weil ich das System sofort stoppen können muss.
- Eine schöne Stimme ist wertlos, wenn Autorität und Privacy unklar bleiben.

Nächste Härtung / nächster Projektschritt

- Öffentliche Demo ohne private Mikrofon-/Vault-Daten bauen.
- Safe Tools und confirmation-required Tools klar trennen.
- Transkript-Audit und Korrekturpfade ergänzen.
- Eine öffentliche Voice-Simulation mit Transkripten statt Live-Mikrofon bauen.
- Safe Tools von bestätigungspflichtigen Tools im UI trennen.

- Audit Trail ergänzen: gehörter Satz, gepasste Absicht, gewähltes Tool, blockierter Befehl, Bestätigung und finale Antwort.

ANIMA Experimental Kernel

Ich wollte spekulative Ideen ueber Kognition nicht als grosse Behauptung stehen lassen, sondern in Code, Tests, Metriken und Evidenzgrenzen uebersetzen.

- Zeigt ambitionierte Forschung mit Disziplin: eine spekulative Idee wurde in Module, Tests, Benchmarks, Methodik und klare Claim-Grenzen übersetzt.
- Kein Beweis für künstliches Bewusstsein; ein ambitioniertes, getestetes Experimentierfeld.

Was ich mir dabei gedacht habe

ANIMA war mein Versuch, eine sehr ambitionierte Idee in Code, Tests und Evidenz zu zwingen. Der Punkt ist nicht, Bewusstsein zu behaupten. Der Punkt ist zu zeigen, dass selbst eine unscharfe Idee in Architektur, Metriken und Grenzen übersetzt werden kann.

Was ich gebaut habe

- Ich kann eine ambitionierte Idee in ein testbares Paket verwandeln.
- 446 lokale Tests wurden im Rebuild erfolgreich ausgeführt.
- Ich begrenze die öffentliche Aussage bewusst: Architektur/Testbed, nicht Bewusstseinsbeweis.

Mechanik im Detail

- ANIMA übersetzt eine spekulative kognitive Architektur in Python-Module, Primitives, State, Temporal Layers, Metrics, Tests, Benchmarks und Methodiknotizen.
- Die stärkste Evidenz ist nicht eine philosophische Behauptung, sondern die Disziplin, eine Idee messbar zu machen und Evidenzstatus-Notizen stärker zu gewichten als Marketing-Sprache.
- Der lokale Lauf mit 446 bestandenen Tests zeigt Umsetzungssubstanz, während die Benchmark-Notizen klar begrenzen, was die Ergebnisse beweisen.

Architektur

- Kernel-, State- und Typmodule bilden den Kern.
- Primitiven wie Qualia, Engram, Valence, Nexus, Impulse, Trace, Mirror und Flux werden testbar gemacht.
- Temporal- und Metrikschichten erlauben Messung statt reiner Philosophie.
- Benchmark- und Evidence-Status-Dateien verhindern überzogene Claims.

Evidenzcluster

- ANIMA-Python-Package mit Kernel, State, Primitives, Temporal Modules, Metrics, Bridge/Shell-Modulen, Tests und Benchmarks.
- Evidenzstatus- und Methodikdateien, die Claims begrenzen und aktuelle Benchmark-Limits benennen.
- Lokaler Rebuild-Testlauf mit 446 bestandenen Tests und einer Konfigurationswarnung.
- Architekturnotizen, die ANIMA mit Memory, zeitlicher Kontinuität, Self-Modeling und Miguel-Integration verbinden.

Claim-zu-Evidenz-Karte

Die öffentliche Version soll stark wirken, weil sie begrenzt ist, nicht weil sie übertreibt.

Claim	Evidenz	Grenze
Zeigt ambitionierte Forschung mit Disziplin: eine spekulative Idee wurde in Module, Tests, Benchmarks, Methodik und klare Claim-Grenzen übersetzt.	ANIMA-Python-Package mit Kernel, State, Primitives, Temporal Modules, Metrics, Bridge/Shell-Modulen, Tests und Benchmarks.; Evidenzstatus- und Methodikdateien, die Claims begrenzen und aktuelle Benchmark-Limits benennen.	Kein Beweis für künstliches Bewusstsein; ein ambitioniertes, getestetes Experimentierfeld.
Die Arbeit ist wertvoll, weil sie Mechanik zeigt, nicht nur Oberfläche.	Kernel-, State- und Typmodule bilden den Kern.; Primitiven wie Qualia, Engram, Valence, Nexus, Impulse, Trace, Mirror und Flux werden testbar gemacht.	Frische öffentliche Demos oder Benchmarks wären die nächste Beweisschicht.

Senior-Level-Signal

Zeigt ambitionierte Forschung mit Disziplin: eine spekulative Idee wurde in Module, Tests, Benchmarks, Methodik und klare Claim-Grenzen übersetzt.

- Ambition mit Zurückhaltung: hoch greifen, ohne künstliches Bewusstsein als bewiesen zu verkaufen.
- Messinstinkt: Benchmarks und Ablationen existieren auch dann, wenn Resultate noch nicht entscheidend sind.
- Evidenzhierarchie: Resultate und Methodik stehen über optimistischer Überblickssprache.

Skeptische Reviewer-Fragen

- Was ist hier wirklich gebaut und was ist noch Design? Antwort: Status und Grenze stehen im Dossier direkt beim Fall ANIMA Experimental Kernel.
- Welche private Evidenz wurde bewusst nicht veröffentlicht? Antwort: lokale Pfade, Credentials, private Kanaldetails und vertrauliche Rohtexte bleiben draußen.
- Warum ist das relevant? Antwort: Der Fall zeigt einen wiederverwendbaren Baustein für Agenten, Tool-Nutzung, Memory, Routing, Validierung oder High-Stakes-Governance.
- Was wäre der nächste belastbare Beweis? Antwort: synthetische Demo, Audit Trace, Benchmark oder externe Review, je nach Fall.

Lernpunkte und Grenzen

Kein Beweis für künstliches Bewusstsein; ein ambitioniertes, getestetes Experimentierfeld.

- Grosse Begriffe brauchen mehr Evidenz als normale Softwareclaims.
- Nicht-signifikante Ergebnisse koennen wertvoll sein, wenn sie die naechste Hypothese formen.
- Die glaubwuerdige Version ist die begrenzte Version.

Nächste Härtung / nächster Projektschritt

- Benchmarkbericht mit Baselines und Statistik sauber öffentlich machen.
- Philosophie, Architektur und gemessene Resultate klar trennen.
- ANIMA als Experimentmodul nutzen, nicht als Hauptclaim.
- Einen sauberen Benchmark-Report mit Baselines, Ablationen und Statistikmethode veröffentlichen.

- Architekturdiagramme von gemessenen Ergebnissen trennen.
- ANIMA als Experimentiermodul für Agent-Memory/Self-Modeling nutzen, nicht als Hauptclaim.

Genesis Workflow System Lineage

Ich zeige Genesis als Vorgeschichte: lokale LLMs, Agenten, Memory, Voice, Parallelisierung, Monitoring und Selbstverbesserungsschleifen. Dadurch wirkt Miguel nicht wie ein einzelner Zufall, sondern wie eine spätere Konsolidierung.

- Zeigt Wiederholung, Evolution und Systemdruck vor Miguel Core.
- Linien-/Entwicklungsevidenz, nicht der stärkste Einzelclaim.

Was ich mir dabei gedacht habe

Genesis ist wichtig, weil es den Weg vor der sauberen Story zeigt. Es war breit, experimentell und teilweise unordentlich, aber genau daraus entstand der Druck, später Miguel Core, Memory-Governance und Routing-Disziplin zu bauen.

Was ich gebaut habe

- A local workflow system lineage around agents, memory, voice, monitoring, parallel execution and self-improvement loops.
- A practical predecessor that made later consolidation into Miguel Core necessary and more intentional.

Mechanik im Detail

- Genesis zeigt, dass Miguel nicht aus dem Nichts kam. Es war ein früheres Workbench-System rund um lokale LLMs, Agents, Memory, Parallelisierung, Monitoring, Voice/Multimodal und Self-Improvement-Loops.
- Die Rolle ist Entwicklungslinie, nicht Hauptclaim. Die frühen Experimente erklären, warum später Konsolidierung, Evidenzgrenzen und Route-Disziplin nötig wurden.

Architektur

- Lokale Modell- und Agentenexperimente.
- Memory- und Workflow-Komponenten.
- Voice- und Kontrollversuche.
- Monitoring- und Verbesserungsschleifen.
- Konsolidierungsdruck Richtung Miguel Core.

Evidenzcluster

- Evidenznotiz zur früheren lokalen KI-Workflow-Linie.

Claim-zu-Evidenz-Karte

Die öffentliche Version soll stark wirken, weil sie begrenzt ist, nicht weil sie übertreibt.

Claim	Evidenz	Grenze
Zeigt Wiederholung, Evolution und Systemdruck vor Miguel Core.	Evidenznotiz zur früheren lokalen KI-Workflow-Linie.	Linien-/Entwicklungsevidenz, nicht der stärkste Einzelclaim.

Claim	Evidenz	Grenze
Die Arbeit ist wertvoll, weil sie Mechanik zeigt, nicht nur Oberfläche.	Lokale Modell- und Agentenexperimente.; Memory- und Workflow-Komponenten.	Frische öffentliche Demos oder Benchmarks wären die nächste Beweisschicht.

Senior-Level-Signal

Zeigt Wiederholung, Evolution und Systemdruck vor Miguel Core.

- Iteration über Zeit statt ein einzelner Zufallstreffer.
- Erkennen, wann Breite konsolidiert werden muss.
- Eine stärkere Lernstory: frühe Komplexität führte später zu klareren Verantwortungsgrenzen.

Skeptische Reviewer-Fragen

- Was ist hier wirklich gebaut und was ist noch Design? Antwort: Status und Grenze stehen im Dossier direkt beim Fall Genesis Workflow System Lineage.
- Welche private Evidenz wurde bewusst nicht veröffentlicht? Antwort: lokale Pfade, Credentials, private Kanaldetails und vertrauliche Rohtexte bleiben draußen.
- Warum ist das relevant? Antwort: Der Fall zeigt einen wiederverwendbaren Baustein für Agenten, Tool-Nutzung, Memory, Routing, Validierung oder High-Stakes-Governance.
- Was wäre der nächste belastbare Beweis? Antwort: synthetische Demo, Audit Trace, Benchmark oder externe Review, je nach Fall.

Lernpunkte und Grenzen

Linien-/Entwicklungsevidenz, nicht der stärkste Einzelclaim.

- Wiederholte Prototypen zeigen Architekturdruck.
- Breite wird erst überzeugend, wenn Evolution und Grenze sichtbar sind.

Nächste Härtung / nächster Projektschritt

- Im Fellowship nur als Linie, im Gesamtportfolio deutlicher zeigen.
- Als Timeline nutzen: Genesis -> Remote/Dual-Brain -> Miguel Core.
- Performance oder Produktionsreife nicht überbehaupten.